Student Name: Jobayer Bhuiyan
Date: Tuesday, February 18, 2026
Course: Network Forensics
Instructor: Christopher Lewis

Abstract

This lab focuses on analyzing multiple packet capture (PCAP) files using command-line tools to identify protocols, authentication activity, and file transfer behavior. Tools such as shasum, capinfos, tshark, and grep were used to verify file integrity, extract metadata, summarize host/protocol activity, and isolate authentication and file transfer indicators.

Across the captures, unencrypted FTP traffic exposed plaintext credentials and a file download request, demonstrating the risk of legacy protocols. HTTP traffic was also reviewed to determine whether downloads occurred and whether authentication attempts were successful. Overall, this exercise demonstrates how repeatable command-line packet analysis supports incident response and evidence-driven conclusions.

Introduction

The purpose of this assignment is to analyze packet capture files from systems suspected of compromise and determine what network activity occurred. Specifically, the analysis aims to (1) verify file integrity, (2) identify active hosts and protocols, (3) determine whether authentication attempts occurred, and (4) confirm whether files were downloaded or uploaded using FTP or HTTP.

Tools and Commands Used

- shasum -c <file>.sha1 (integrity verification)
- capinfos <file>.pcap (capture metadata)
- tshark -r <file>.pcap -q -z ip_hosts,tree (host/IP statistics)
- tshark -r <file>.pcap -q -z io,phs (protocol hierarchy statistics)
- tshark display filters (e.g., ftp, http, ftp.request.command == RETR)
- grep for indicators (e.g., USER/PASS, GET/POST, PUT)

Analysis and Findings

1. ShellLab.pcap Results

1.1 File Integrity and Metadata

The SHA1 verification for ShellLab.pcap returned OK, confirming the capture file was not modified after creation. Using capinfos, the capture duration and timestamp range were recorded for scoping the analysis window.

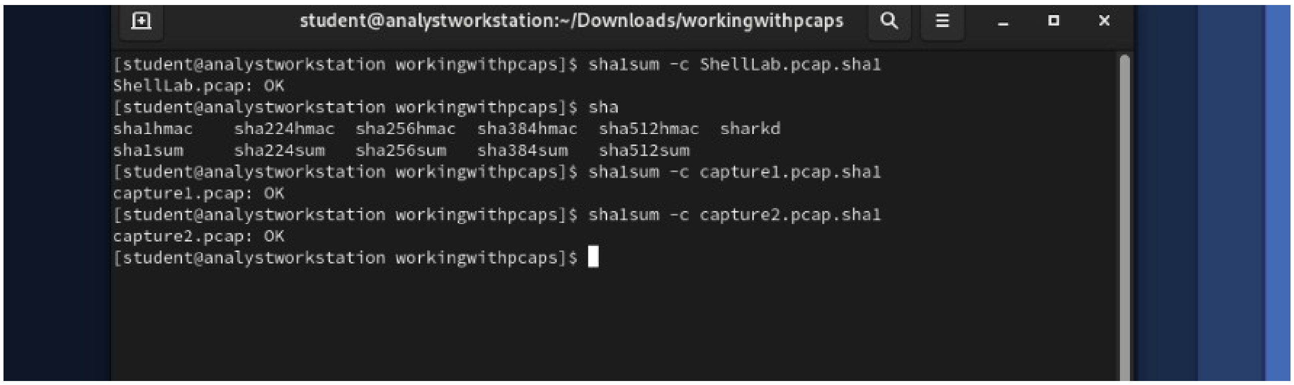

Figure 1.1. SHA1 verification for ShellLab.pcap showing OK status for file integrity.

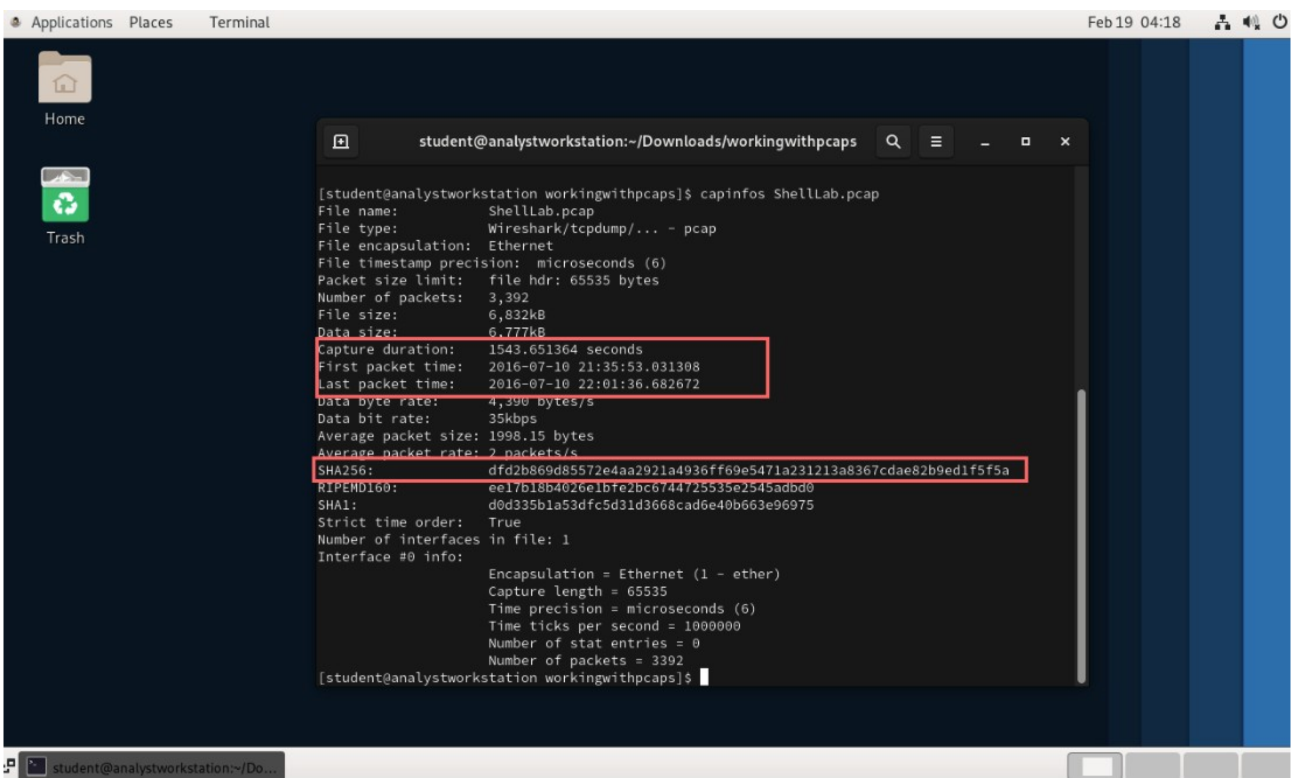

Figure 1.2. capinfos output for ShellLab.pcap showing capture duration and timestamp range.

1.2 Host and Protocol Identification

IPv4 host statistics show the dominant internal host was 172.31.21.16 with the highest packet count. Protocol hierarchy statistics indicate a mix of DNS, HTTP, TLS, SMTP, and FTP/FTP-DATA traffic, with FTP present as a notable plaintext service.

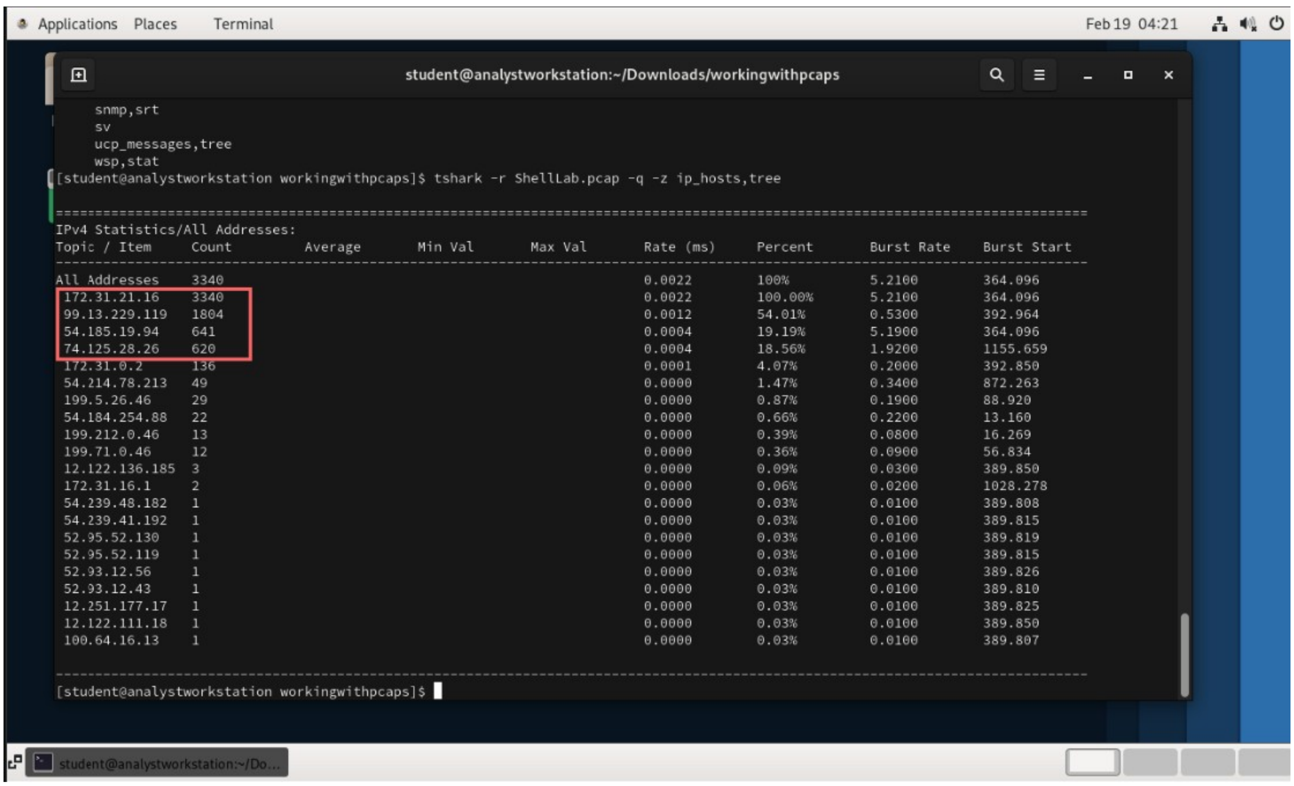

Figure 1.3. Host/IP statistics (ip_hosts,tree) for ShellLab.pcap highlighting top talkers.

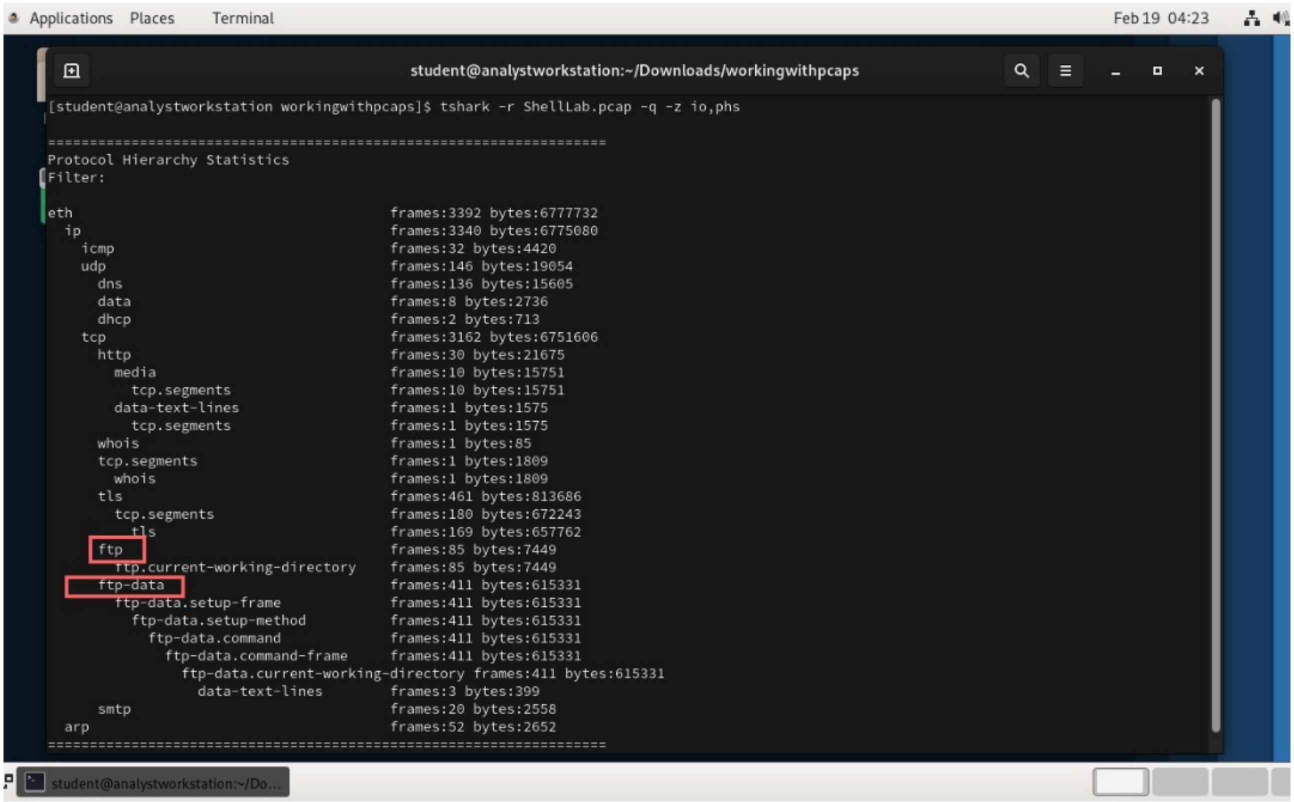

Figure 1.4. Protocol hierarchy statistics (io,phs) for ShellLab.pcap highlighting FTP and FTP-DATA.

1.3 FTP Activity and File Transfer Indicators

FTP traffic was isolated and written to a separate pcap for focused analysis. A RETR command was observed for the file xfil.rar, indicating a file download from the FTP server. No STOR commands were observed, suggesting no file uploads occurred over FTP during the capture.

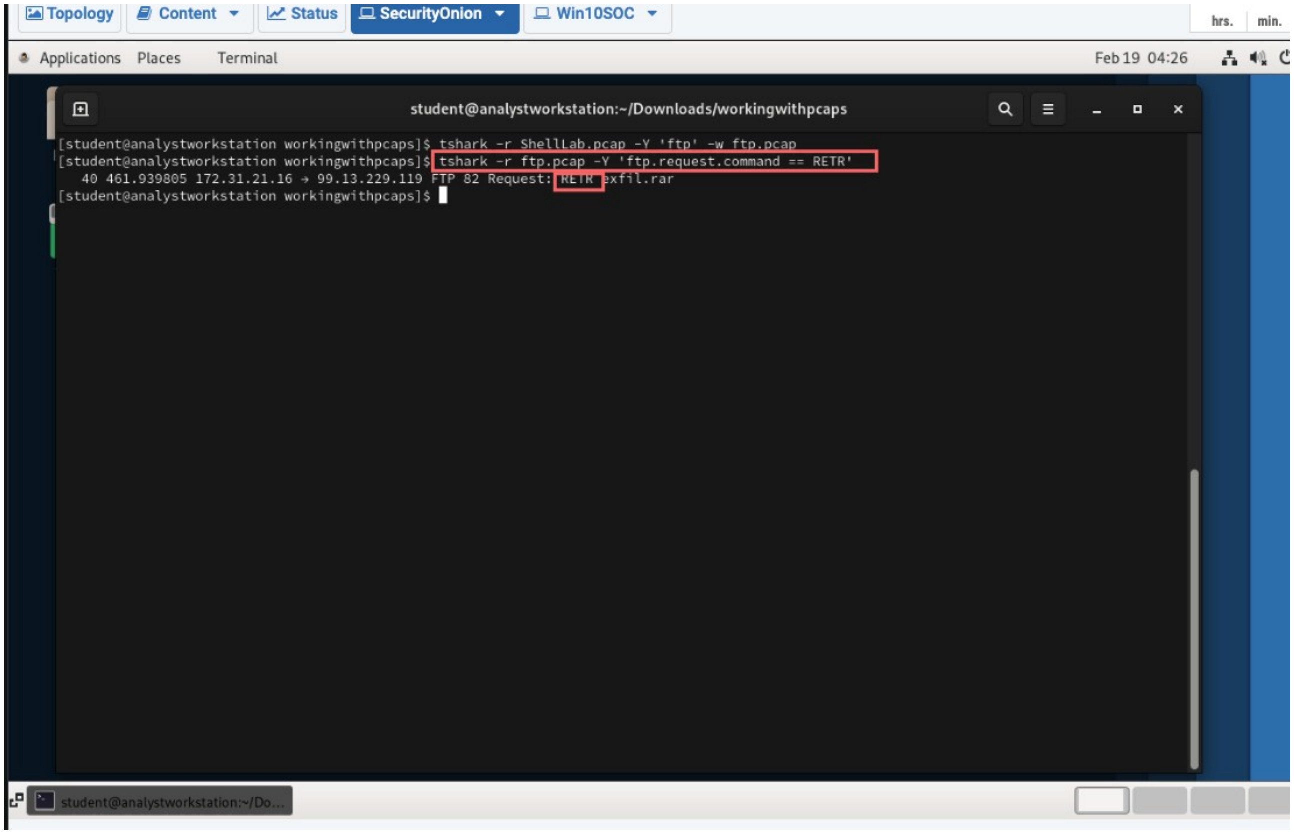

Figure 1.5. Filtering ShellLab.pcap for FTP traffic and identifying a RETR request for xfil.rar.

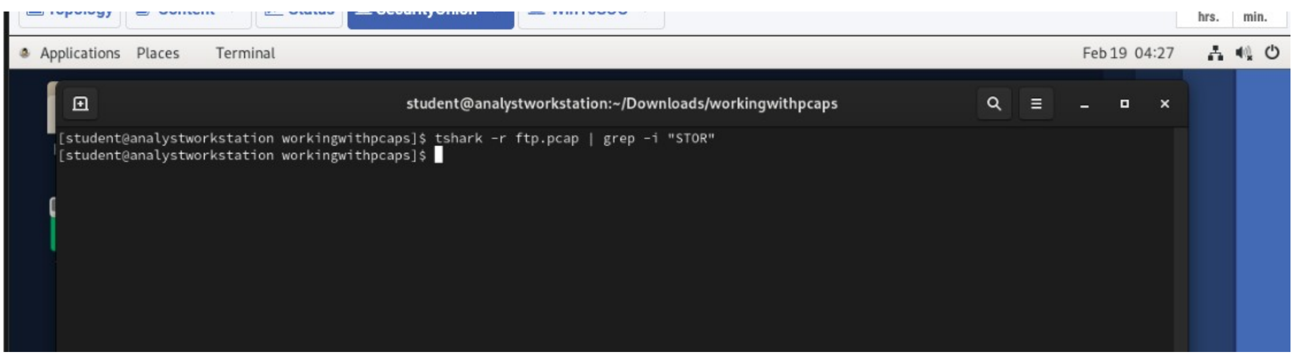

Figure 1.6. Searching for FTP STOR commands returned no results, indicating no FTP uploads.

1.4 FTP Authentication Evidence

Filtering for USER and PASS commands revealed plaintext credentials transmitted in the FTP session. Both failed and successful login attempts were present. Successful authentication was identified for:

• Username: ninja

• Password: deadbeefbabeface
A failed login attempt was also observed for ninja using the password treasure123.

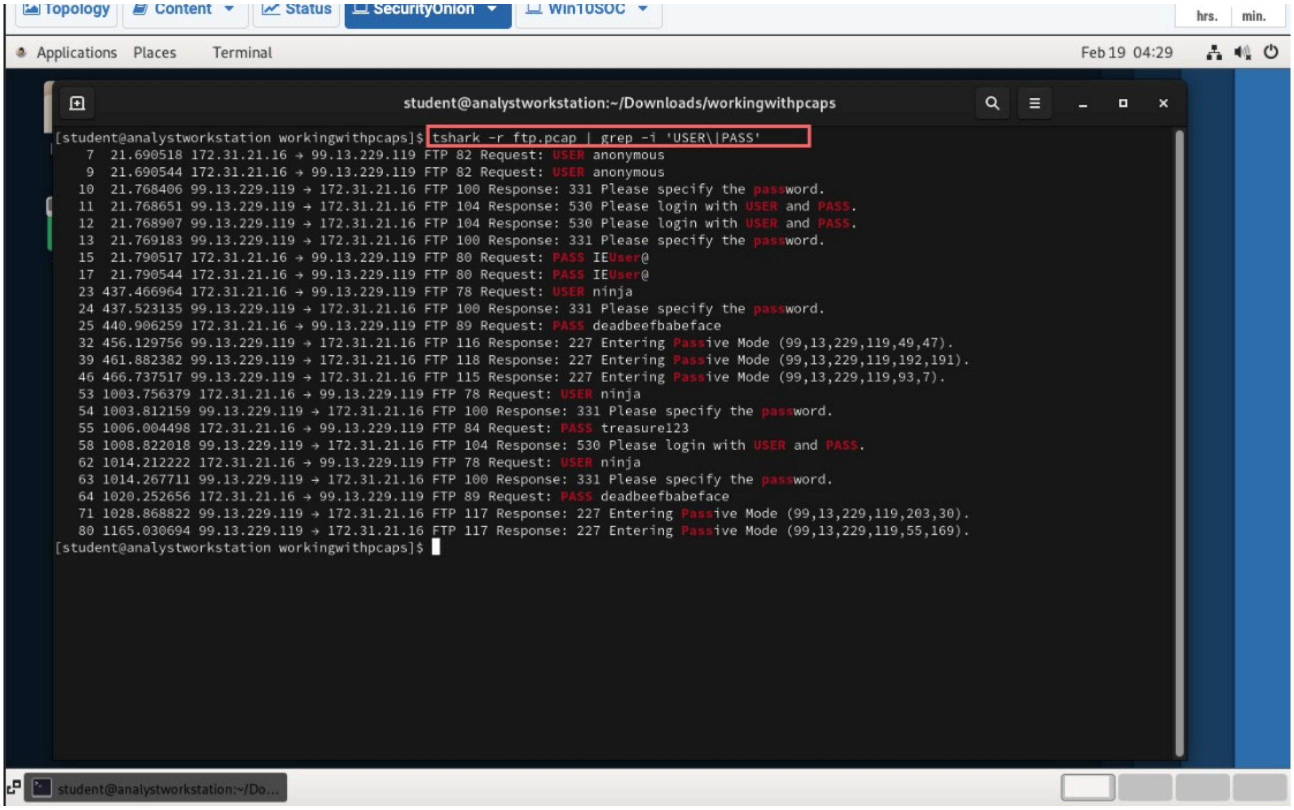

Figure 1.7. Plaintext FTP authentication attempts (USER/PASS) extracted from ftp.pcap.

1.5 HTTP Request Review

HTTP GET requests were observed to multiple hosts, including package repositories and web resources. A search for HTTP PUT activity returned no evidence of file uploads within the provided output, supporting the conclusion that HTTP traffic in this capture primarily consisted of downloads/requests rather than uploads.

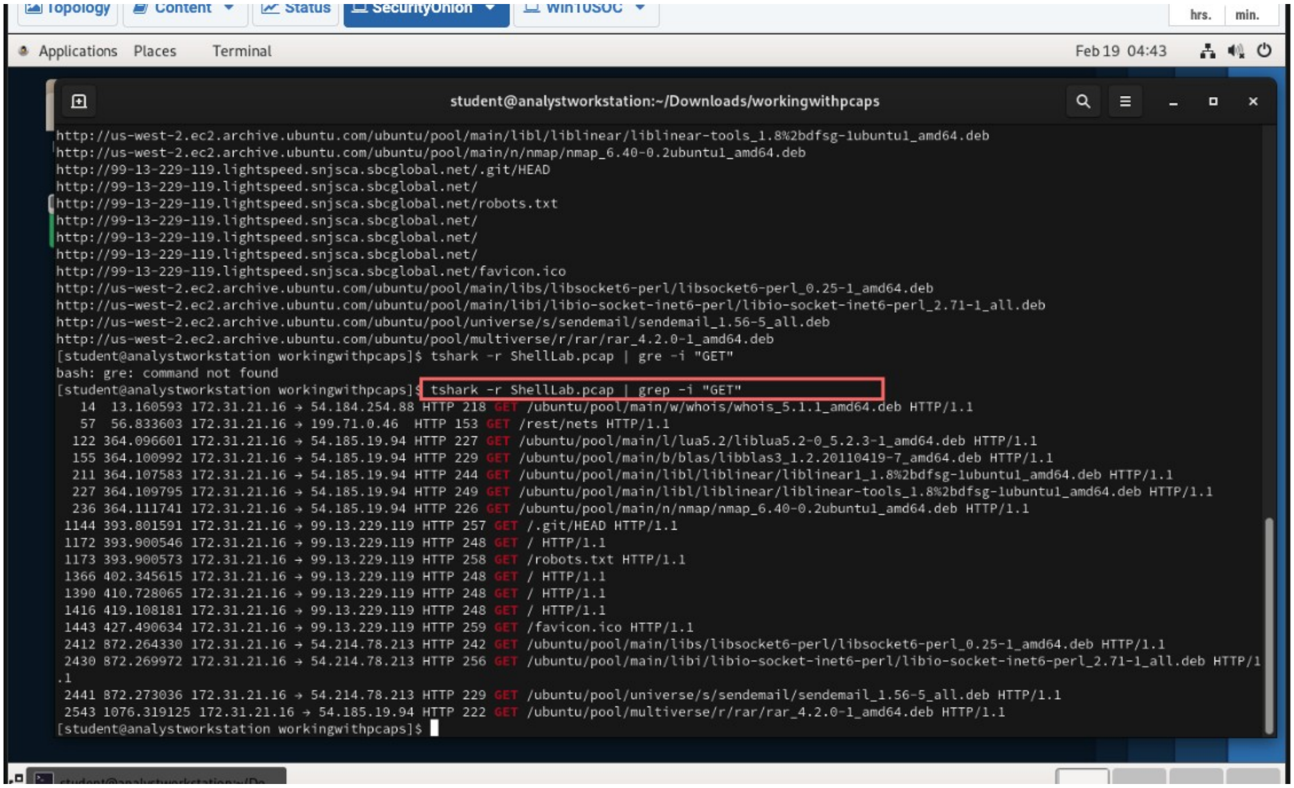

Figure 1.8. HTTP GET requests extracted from ShellLab.pcap using tshark and grep.

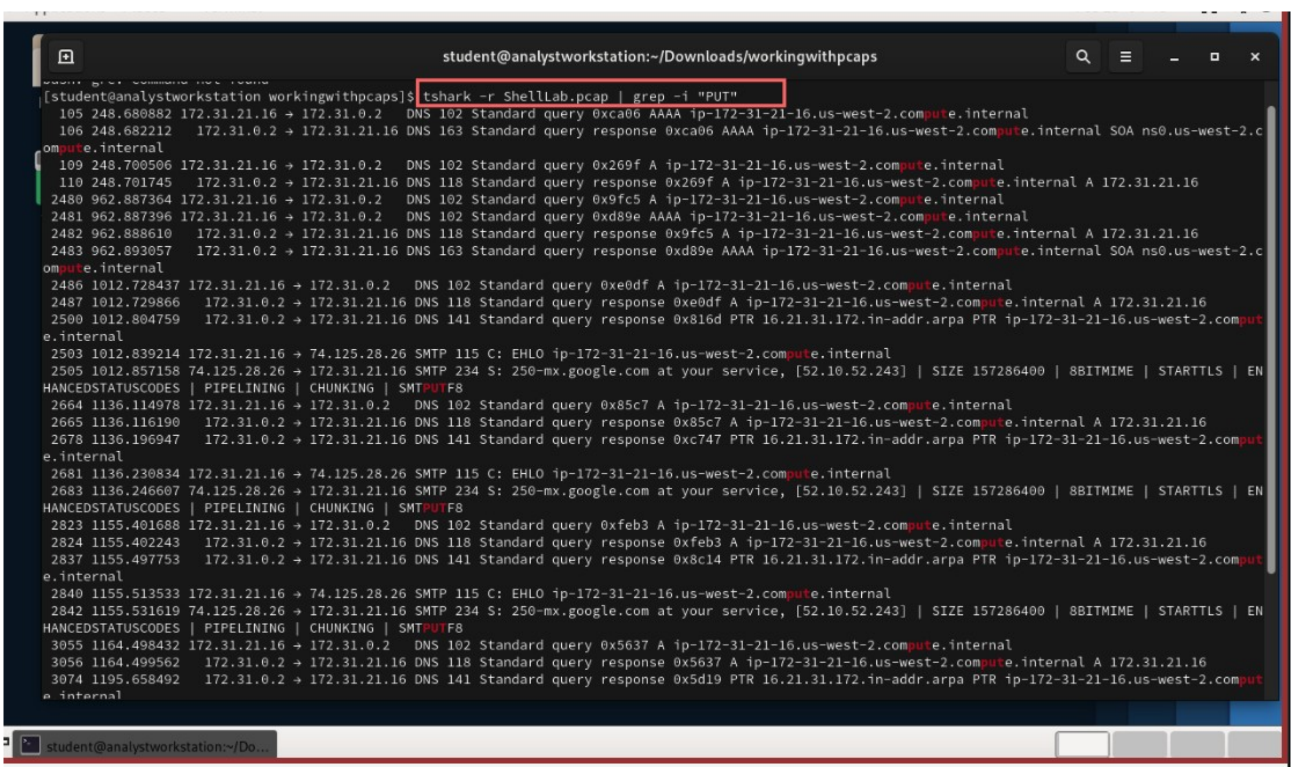

Figure 1.9. Searching for HTTP PUT indicators; output shows no HTTP PUT uploads identified.

2. Capture1.pcap Results

2.1 File Integrity and Metadata

SHA1 verification for capture1.pcap returned OK. capinfos shows a short capture window (~33 seconds) with 166 packets, which limits the scope of observable activity but is sufficient to identify high-level protocols.

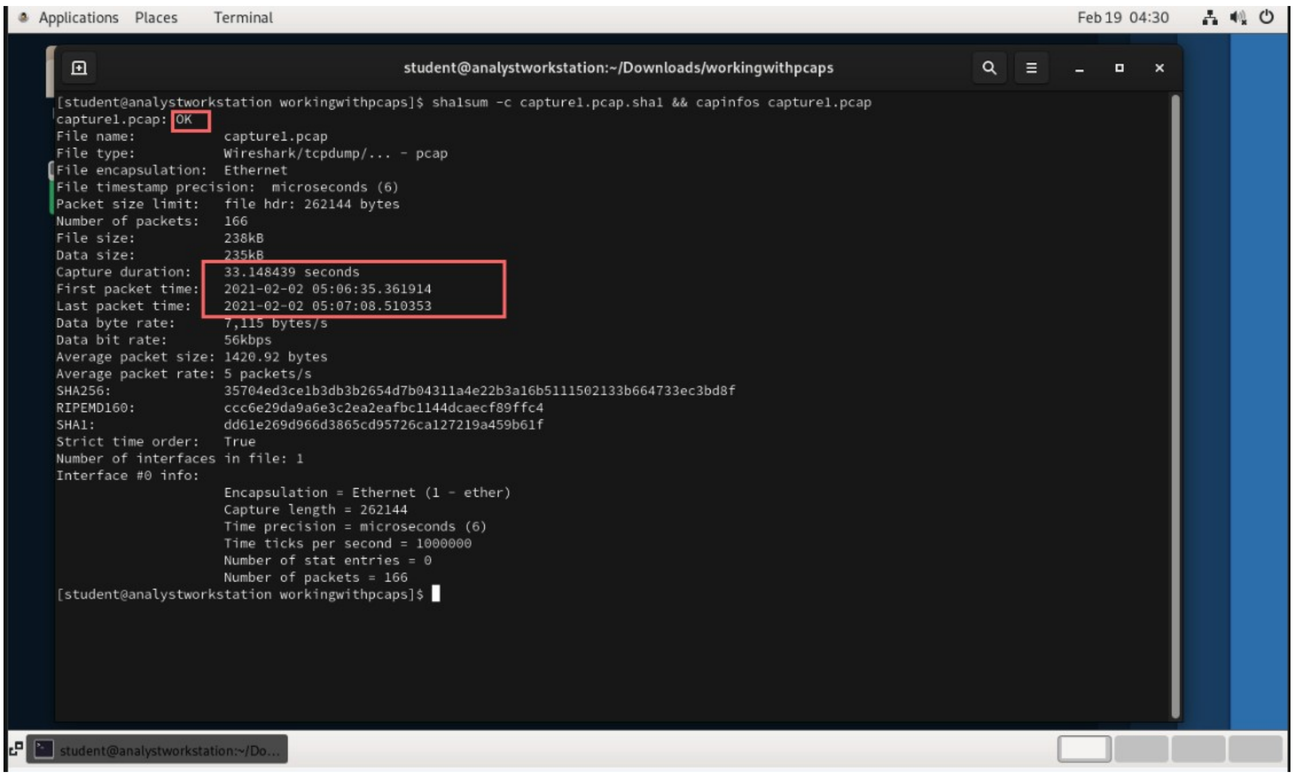

Figure 2.1. SHA1 verification (OK) and capinfos metadata for capture1.pcap (packet count and time range).

2.2 Host and Protocol Identification

Host statistics show the capture is dominated by the internal host 172.31.0.2 communicating primarily with 102.129.202.163. Protocol hierarchy statistics indicate IP/TCP with limited HTTP activity and an image/jpeg artifact (image-jfif), suggesting web-based content retrieval.

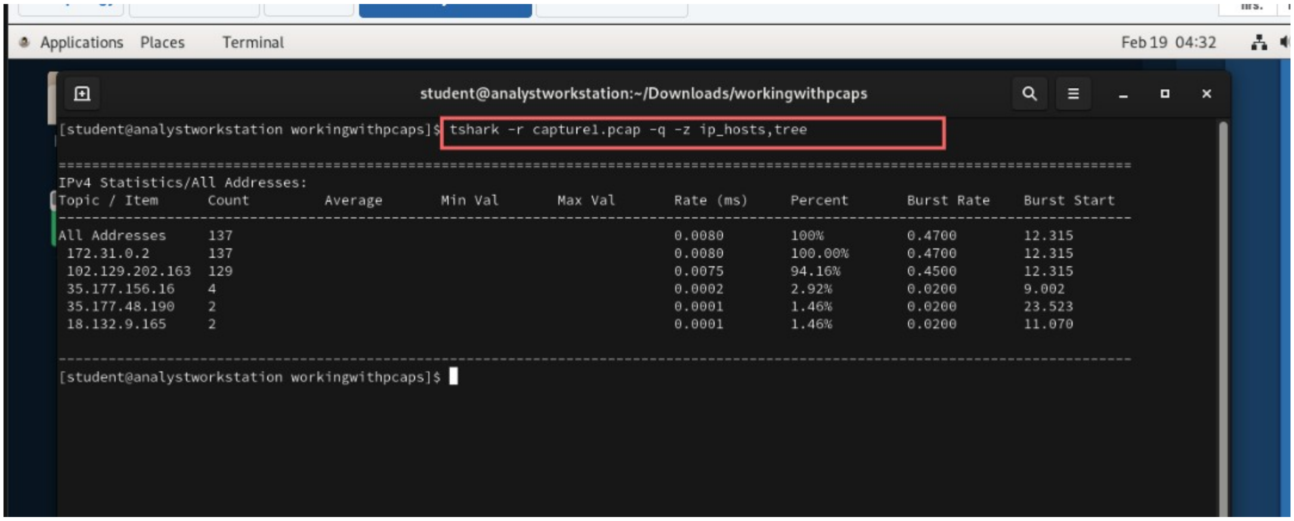

Figure 2.2. Host/IP statistics (ip_hosts,tree) for capture1.pcap highlighting the primary hosts.

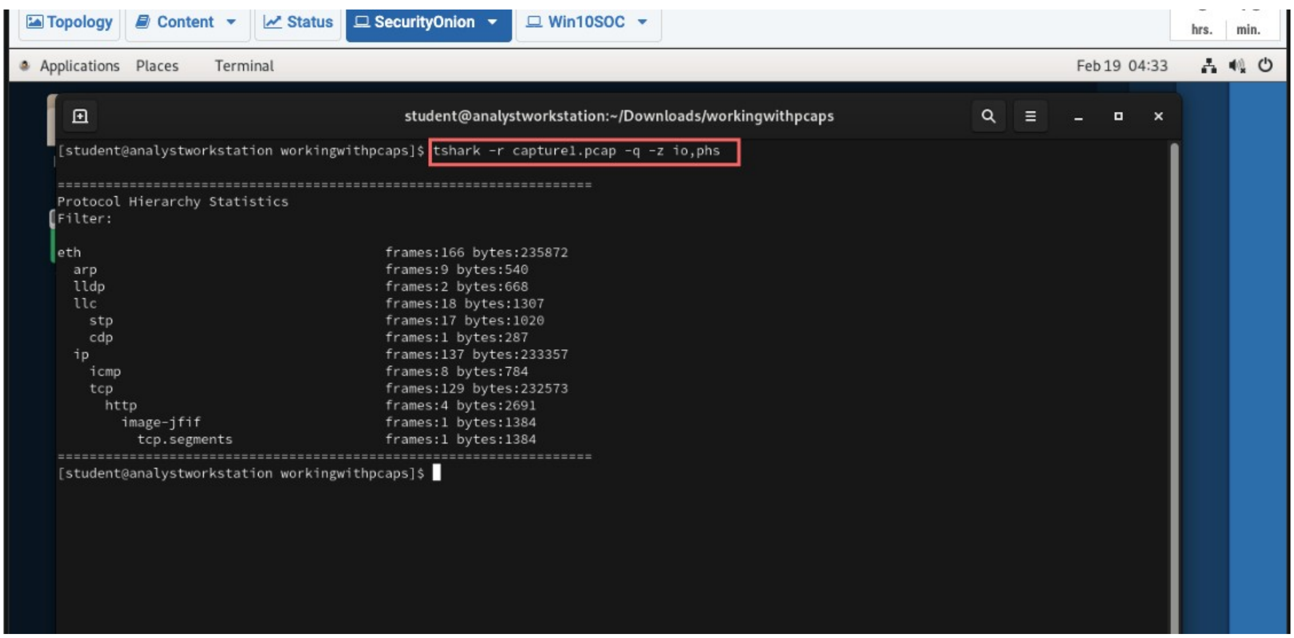

Figure 2.3. Protocol hierarchy statistics (io,phs) for capture1.pcap showing IP/TCP/HTTP and image-jfif.

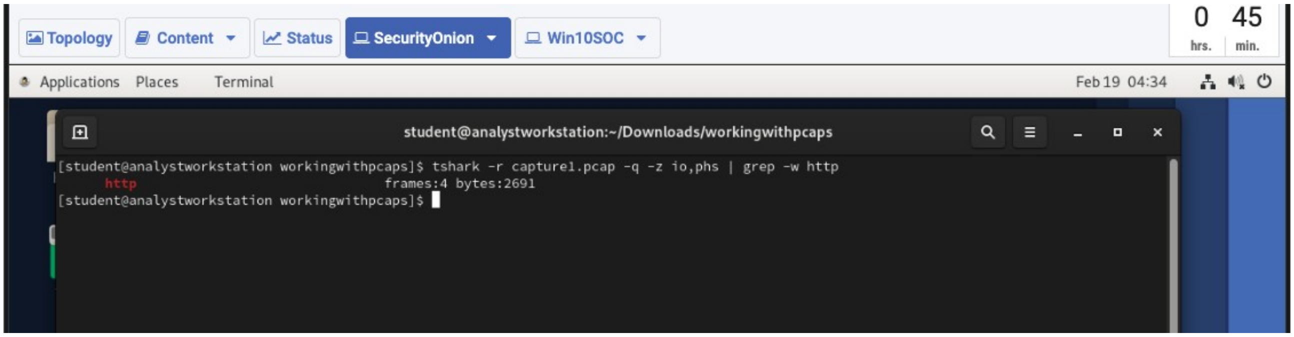

Figure 2.4. HTTP presence confirmed (frames count) by filtering protocol hierarchy output for HTTP.

2.3 File Transfer Assessment (HTTP)

Based on the limited HTTP frames and the presence of image-jfif, this capture appears to contain a small amount of web browsing or content retrieval. No evidence of HTTP uploads is shown in the provided output.

3. Capture2.pcap Results

3.1 File Integrity and Metadata

SHA1 verification for capture2.pcap returned OK. capinfos indicates a longer capture window (~412 seconds) with 11,331 packets, allowing stronger conclusions about host behavior and repeated HTTP activity.

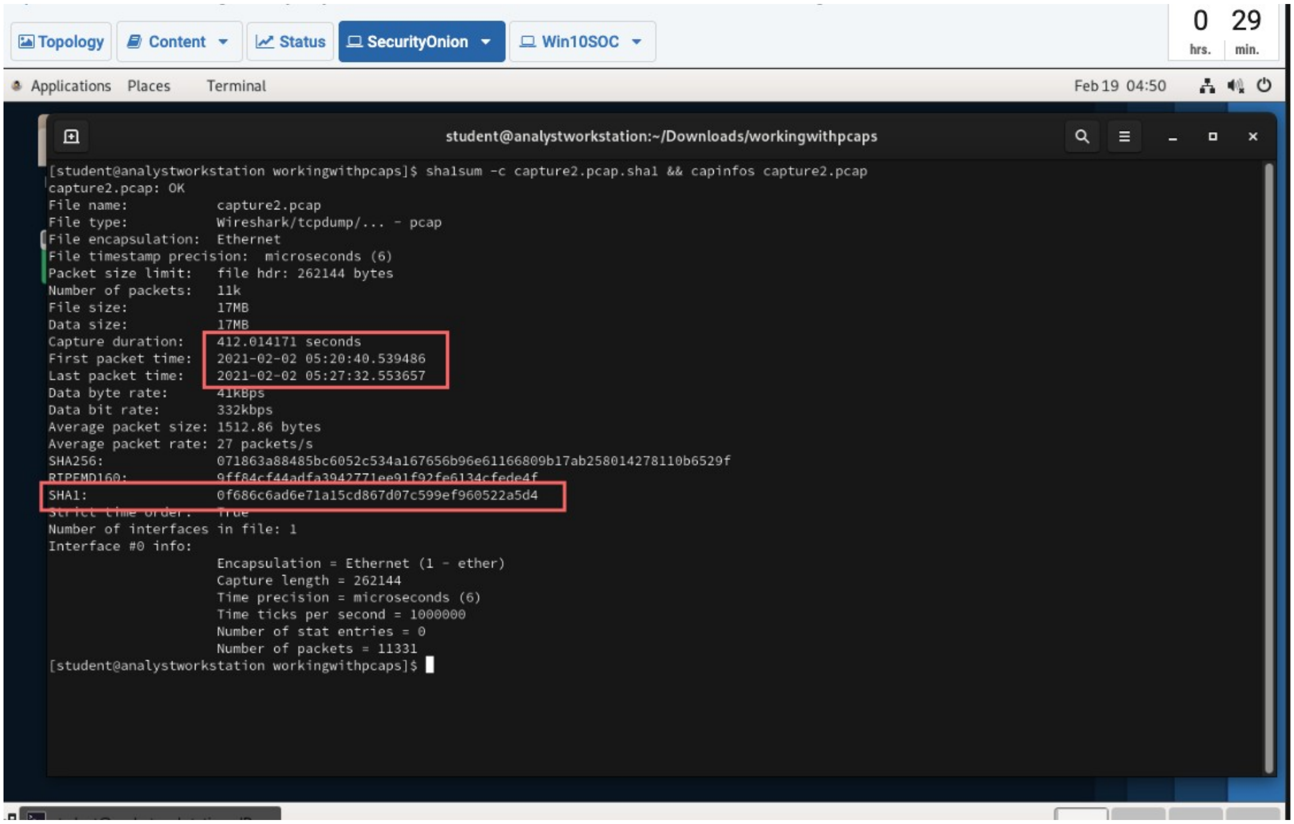

Figure 3.1. SHA1 verification (OK) and capinfos metadata for capture2.pcap (packet count and time range).

3.2 Host and Protocol Identification

Host statistics show sustained communication between 172.31.0.2 and the external IP 193.176.211.47. Protocol hierarchy statistics confirm TCP-dominant traffic with minimal HTTP frames.

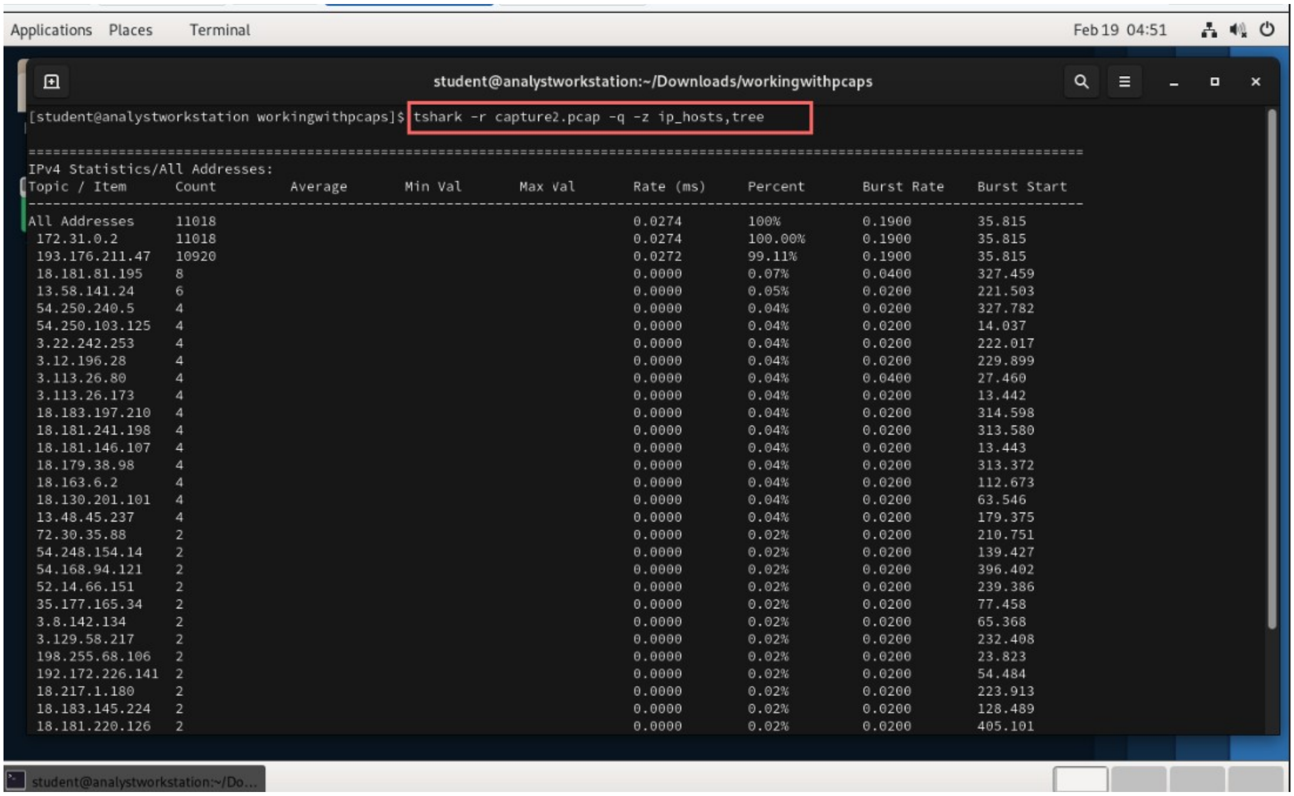

Figure 3.2. Host/IP statistics (ip_hosts,tree) for capture2.pcap highlighting the dominant peer hosts.

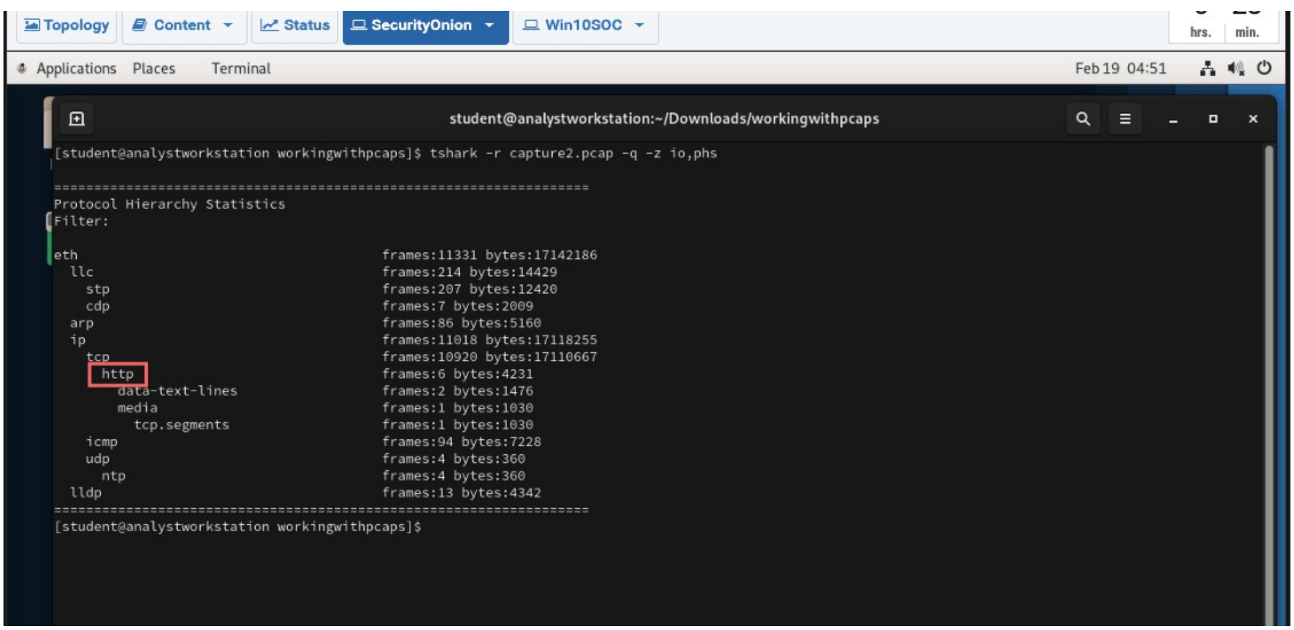

Figure 3.3. Protocol hierarchy statistics (io,phs) for capture2.pcap showing minimal HTTP traffic.

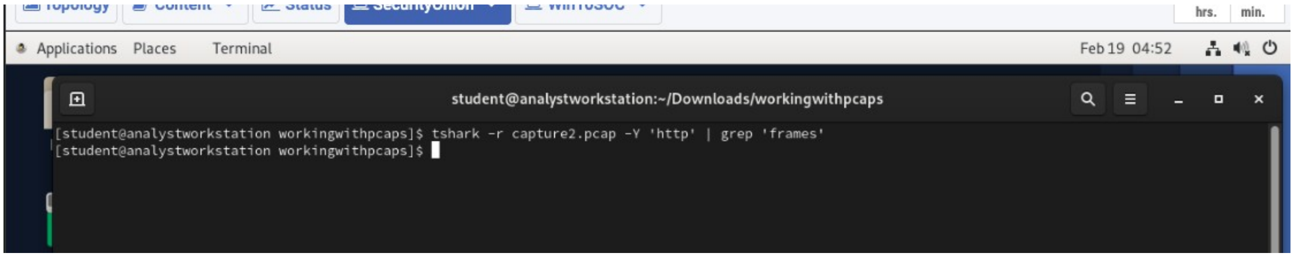

Figure 3.4. Direct HTTP filtering returned no additional frame summary output (limited HTTP activity).

3.3 HTTP Download Attempts (GET/POST)

HTTP traffic was extracted into a separate file for review. Multiple HTTP GET requests were observed for /thegoods/leetvid.zip. No POST requests were identified, indicating the activity was consistent with download attempts rather than uploads.

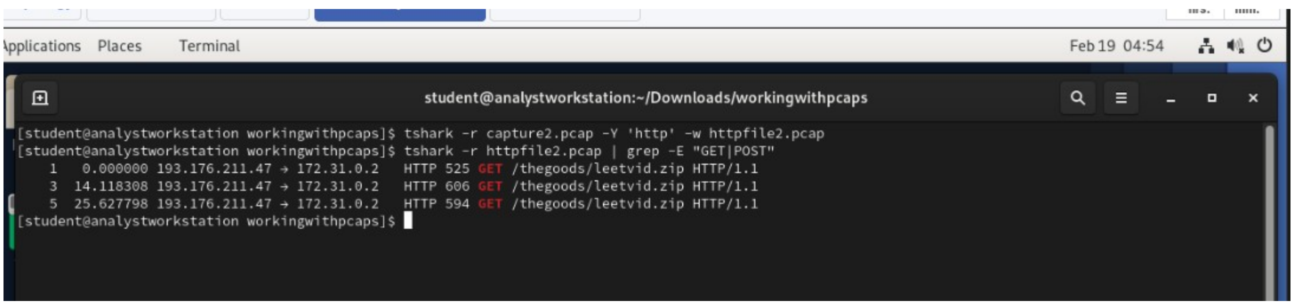

Figure 3.5. HTTP extraction and GET/POST review showing repeated GET requests for leetvid.zip and no POST.

3.4 HTTP Authentication Outcome

No http.authorization headers were identified in the extracted HTTP traffic. Additionally, the HTTP responses observed were 401 Unauthorized, indicating authentication failed and the requested resource was not successfully retrieved (no 200 OK responses were observed in the provided output).

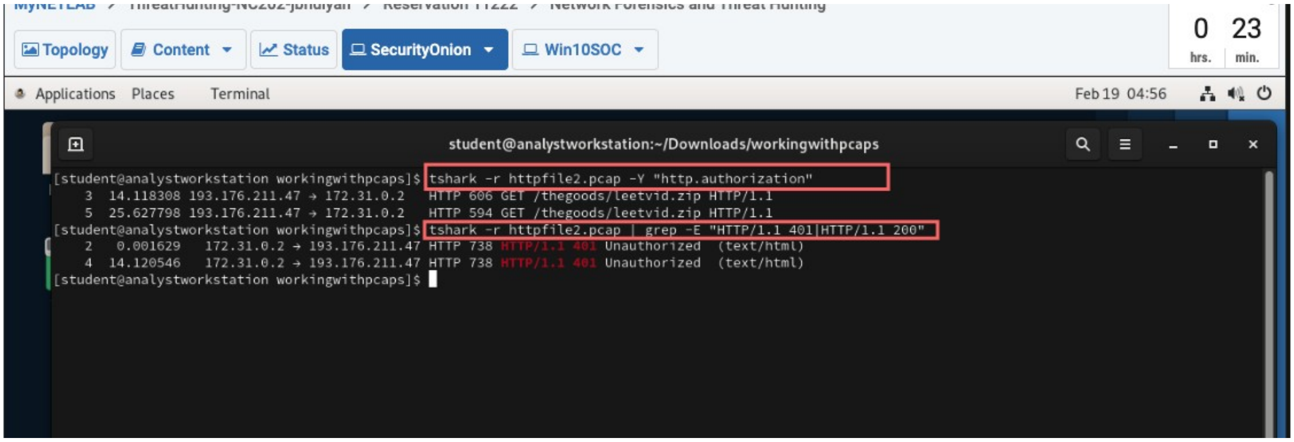

Figure 3.6. Authentication check shows no http.authorization and responses of HTTP/1.1 401 Unauthorized.

Conclusion

Across the analyzed captures, command-line packet inspection revealed clear indicators of protocol usage, authentication attempts, and file transfer behavior. ShellLab.pcap contained plaintext FTP authentication and a RETR request for xfil.rar, demonstrating the exposure created by unencrypted FTP. Capture1.pcap showed brief TCP/HTTP activity consistent with limited web content retrieval. Capture2.pcap contained repeated HTTP GET attempts for leetvid.zip, but response codes indicate authentication failures (401), supporting the conclusion that the download was not successfully completed.

Overall, the lab reinforced that rapidly scoping captures with capinfos and tshark statistics, then pivoting into protocol-specific filters, provides an efficient and defensible workflow for network forensics.

References

- Wireshark/tshark documentation (Wireshark User’s Guide and tshark man page).
- capinfos man page (Wireshark suite).
- GNU coreutils shasum/sha1sum documentation.